Assignment 3 (C): Reverse a String

CS 101 Programming Principles · Fall 2026

LANGUAGE	FILE TO WRITE	DUE
C (standard library only)	`reverse.c`	**Nov 20, 2026, 11:59 PM**
POINTS	YOU SUBMIT	TIME
20 (demo course)	`reverse.c` **+ your** `.codelapse` **recording**	**About 45–60 minutes, then up to 1 hour for 4 short questions**

1. Overview

You will write a small C program with two functions: one that reverses a string **in place**, and one that counts its vowels. It is a short program, but it touches several ideas that matter in every C program you will write: how strings are stored, how loops move through an array, and what it means for a function to change the data it is given.

You will write it in VS Code while the **CodeLapse** extension records how you work. When you submit, you will answer four short questions about your own work and your own code.

What you will practise

- How C stores a string: an array of `char` that ends with the null character `'\0'`.
- Using `strlen` and loops with indices (or pointers) to walk through an array.
- Changing data **in place** through a pointer, and protecting data you only read with `const`.
- Testing your own code on ordinary and unusual inputs, and fixing compiler errors and warnings.

2. What to build

Write everything in one file, `reverse.c`. Use only the C standard library (for example `<stdio.h>`, `<string.h>`, `<ctype.h>`).

`void reverse(char *s)`

- Reverses the string `s` **in place**: after the call, the same array holds the characters in reverse order.
- Must **not** use a second array or string, and must not allocate memory (`malloc`). A single `char` variable for swapping is fine.
- Must work for any string, including the empty string `""` and a string of one character.
- Returns nothing: the change happens to the caller's array.

`int count_vowels(const char *s)`

- Returns how many vowels the string contains. The vowels are `a e i o u`, in **either case** (`A E I O U` count too).
- `y` is **not** a vowel for this assignment. Spaces, digits and punctuation are not vowels.
- Must not change the string. The `const` in the parameter is part of the required signature.

`int main(void)`

- Puts a test word in a `char` **array** (for example `"codelapse"`), calls both functions on it, and prints the reversed word and the vowel count.
- Returns `0`.

> **Not allowed**
> A second array or buffer to build the reversed string; `malloc`; any non-standard library; library functions that reverse for you (some systems have `strrev`, which is not standard C). The program must compile with **no warnings** using `gcc -Wall`.

3. Examples

Every row was checked by running a correct solution. The quotes only show where the string starts and ends; they are not part of it.

INPUT	AFTER `REVERSE`	`COUNT_VOWELS`	WHY IT IS A USEFUL TEST
`"codelapse"`	`"espaledoc"`	4	The normal case (odd length: 9)
`""`	`""`	0	Empty string: nothing to swap, must not crash
`"a"`	`"a"`	1	One character: stays the same
`"ab"`	`"ba"`	1	Shortest even length
`"abc"`	`"cba"`	1	Short odd length: the middle stays put
`"racecar"`	`"racecar"`	3	A palindrome reads the same reversed
`"a b"`	`"b a"`	1	Spaces are characters too
`"Hello, World!"`	`"!dlroW ,olleH"`	3	Punctuation, a space, and capitals
`"Programming"`	`"gnimmargorP"`	3	A capital letter moves to the end
`"AEIOU aeiou"`	`"uoiea UOIEA"`	10	Upper- and lower-case vowels both count
`"rhythm"`	`"mhtyhr"`	0	`y` is not a vowel

Example run

```
$ gcc -Wall -o reverse reverse.c
$ ./reverse
Original: codelapse
Reversed: espaledoc
Vowels: 4
```

The exact wording of the output is up to you, but it must show the reversed word and the vowel count. Reversing does not change how many vowels a word has, so you can count before or after reversing.

4. A plan that works

1. Create the folder, open it in VS Code and start CodeLapse (section 8). Create `reverse.c`.
2. Write a `main` that only prints your test word. Build and run it, so you know your setup works.
3. Write `reverse`. Call it from `main`, print the result, build and run.
4. Try the examples above by changing the test word (or by testing several words in `main`).
5. Write `count_vowels`. Print its result and check it against the table.
6. Build with `-Wall` one last time and fix every warning. Save the file.

5. Hints

Read one hint at a time and try it yourself before moving on to the next. They start general and get more specific.

1 GETTING STARTED

A C string is an array of `char` followed by `'\0'`. `strlen(s)` (from `<string.h>`) gives the number of characters **before** the `'\0'`, so the last real character is at index `strlen(s) - 1`. Never move the `'\0'`.

2 THE IDEA

Picture two fingers, one on the first character and one on the last. Swap what they point at, then move both fingers one step towards the middle. Repeat.

3 WHEN TO STOP

Stop when the fingers meet or cross: that is after about half the length. If you keep going to the end, every pair is swapped a second time and the string comes back unchanged. With an odd length the middle character never needs to move. Check your stopping rule against `"ab"` and `"abc"`.

4 SWAPPING

To swap two characters you need somewhere to keep one of them for a moment: one temporary `char`. If you write `s[i] = s[j]; s[j] = s[i];` without it, both end up the same.

5 INDICES OR POINTERS

You can use two `int` or `size_t` indices, or two `char *` pointers. Both are fine; be ready to explain your choice. Careful with the empty string: `strlen` returns `size_t`, which cannot be negative, so `strlen(s) - 1` on `""` is a huge number, not `-1`. Make sure your loop does not run at all for `""`.

6 COUNTING VOWELS

Walk through the string until you reach `'\0'` and check each character. For upper case, either compare against both cases or turn the character into lower case first with `tolower` from `<ctype.h>` (cast the argument to `unsigned char`). If you look characters up in a string of vowels with `strchr`, remember that `strchr` also "finds" the `'\0'` at the end, so only call it on real characters.

7 CONST

`const char *s` promises that `count_vowels` only reads the string. If you try to change a character through `s`, the compiler reports an error. That is the point.

8 **MAIN**

Write `char word[] = "codelapse";`, not `char *word = "codelapse";`. The first makes an array you are allowed to change; the second points at a string literal, and changing it is undefined behaviour (often a crash). Because `reverse` changes `word` itself, print the original **before** you reverse it.

6. Test your program

- Build with `gcc -Wall -o reverse reverse.c` after every change, in the VS Code terminal. Read warnings as carefully as errors.
- Run `./reverse` and compare with the table. Change the test word in `main` to try the edge cases, or call your functions on several words and print each result.
- A quick way to check yourself: `printf("%s %d\n", word, count_vowels(word));` after each step.

Self-check before you submit

- [ ] Compiles with `gcc -Wall` with no errors and no warnings.
- [ ] `reverse` uses no second array and no `malloc`.
- [ ] Correct for `""`, `"a"`, `"ab"`, `"abc"` and `"Hello, World!"`.
- [ ] `count_vowels` counts capital vowels and does not count `y`.
- [ ] The function signatures are exactly `void reverse(char *s)` and `int count_vowels(const char *s)`.
- [ ] `main` prints the reversed word and the vowel count, and returns `0`.
- [ ] You can explain, in your own words, why your loop stops where it does.

7. Common mistakes

MISTAKE	WHAT HAPPENS, AND THE FIX
Looping all the way to the end	Every pair is swapped twice and the string looks unchanged. Stop at the middle.
Off by one at the end	Using `s[len]` as the last character swaps the `'\0'` to the front, and the string looks empty. The last character is `s[len - 1]`.
Swapping without a temporary	Both positions end up with the same character.
Changing a string literal	`char *w = "hello"; reverse(w);` may crash. Use `char w[] = "hello";`.
Forgetting a semicolon or a bracket	The compiler often reports it on the line **after** the mistake. Look at the line above too.
Forgetting capital vowels, or counting `y`	`"AEIOU aeiou"` must give 10 and `"rhythm"` must give 0.
Printing the "original" after reversing	The array has already changed, so both lines show the reversed word.
Ignoring warnings	Warnings from `-Wall` often point at real problems, such as an unused variable. Calling `strlen` without `#include <string.h>` gives a warning or an error, depending on the compiler.

8. Working with CodeLapse

CodeLapse records how your program grows, so the questions can be about *your* work. Set it up before you start writing. The full tester guide (installation and troubleshooting) is in `docs/TESTER_GUIDE.md` of the CodeLapse repository.

1. **Check your tools.** VS Code 1.93 or newer, and a C compiler: `gcc --version` in a terminal should print a version (on a Mac it may say "clang"; that is fine).
2. **Install CodeLapse.** Download `codelapse-0.1.0.vsix` from
   https://github.com/Shanthan2307/Codelapse_ext/releases/tag/v0.1.0-pilot
   In VS Code: **Extensions** → `···` → **Install from VSIX…** and pick the file. A short walkthrough opens.
3. **Choose your folder.** Create an empty folder (for example `reverse-assignment`), open it with **File** → **Open Folder…**, then click **CodeLapse: Choose folder** in the status bar and pick it. The status bar now shows the folder name with a record icon. Only this folder is recorded.
4. **Write and build in VS Code.** Create `reverse.c`, and build and run in the VS Code terminal (**Terminal** → **New Terminal**, bash, zsh or PowerShell): `gcc -Wall -o reverse reverse.c` then `./reverse`. Builds in a separate terminal app are not recorded. Failed builds are fine and normal.
5. **Export.** Open the Command Palette (`Cmd+Shift+P` / `Ctrl+Shift+P`) and run **CodeLapse: Export Assignment Bundle (.codelapse)**. Save the `.codelapse` file.
6. **Submit.** Click **Open Submission Page** in the message that appears (or run **CodeLapse: Open Submission Page**). The first time, VS Code asks for the address your instructor gives you. On the page, pick your name, add the `.codelapse` file and `reverse.c`, and click **Submit Assignment**.

9. The four questions

Right after you submit, the page shows **four short questions about your own work**: two about things you did while working (for example a build that failed and how you fixed it, or something you rewrote), and two about specific lines of your code. Every student gets different questions.

- **You have one hour from submitting** to answer, and you can submit your answers **once**. A timer shows the time left; your draft is kept if you refresh the page.
- Write a few sentences each, **in your own words**. Spelling and grammar do not matter.
- Explain **why**, not just what: why the loop stops there, why you chose that approach, what the compiler error told you.
- Be specific: mention the actual line, variable, error or change. "It works" says much less than "it stops at half the length because …".
- If you are not sure, say what you think and why. An honest partial answer is better than a vague one.

10. Working honestly, and what is recorded

Write the code yourself, and please turn off AI coding assistants (such as GitHub Copilot) for this assignment. Mistakes, failed builds and rewrites are completely normal; they are part of how programs get written.

CodeLapse records only the folder you choose: what you type, when you switch files, and your builds with their output. It does not record other folders, websites, other apps or your file paths and username. Nothing leaves your computer until you export and upload it yourself.

Your instructor sees your code, your answers, a summary of your recording (how long you worked, how much was typed or added in larger pieces such as pastes, how your builds went), and a short review of how well your

answers explain your code. None of this is a grade or an accusation: it describes how the work happened, and your instructor decides what it means. If you are unsure whether something is allowed, ask your instructor.

11. If something goes wrong

WHAT YOU SEE	WHAT TO DO
`gcc: command not found`	Mac: run `xcode-select --install`. Windows: ask your instructor which compiler to install.
Status bar says **CodeLapse: Choose folder** or **Not recording**	Click it and choose your assignment folder.
Builds don't seem to be recorded	Build in the VS Code terminal (bash, zsh or PowerShell), not a separate Terminal app.
"Choose who you are before submitting."	Pick your name at the top of the form.
"Attach your .codelapse recording."	Add the `.codelapse` file you exported.
"The recording has no sessions in it."	Nothing was recorded in that folder. Check you chose the right folder.
"… is not a C source file. …"	Upload only `reverse.c` (not the compiled `reverse` program).
"Your questions could not be prepared."	Nothing to do: your submission was received and your instructor has been told.

Anything else: ask your instructor. Good luck, and have fun with it.